Chapter 1

Wednesday, November 19, 2025 10:21 AM

Unit 1: Language Preliminaries (8 Hrs.)

- **Introduction to .NET Framework**
  (no qn asked)
- **Compilation and execution of .NET applications**
  (no qn asked)
- **Basic Language Constructs, Constructor, Properties, Arrays, and String**
  (no qn asked)
- **Indexers**
  - **PYQ:** What are indexers in C#? How are they different from properties? Explain with an example.
- **Inheritance and use of "base" keyword**
  (no qn asked)
- **Method Hiding and Overriding**
  - **PYQ:** Method hiding vs Method overriding
- **Applying Polymorphism in Code Extensibility**
  - **PYQ:** Polymorphism
- **Structs and Enums**
  - **PYQ:** Differentiate between struct and enum. Why do we need to handle exceptions? Illustrate with an example with your own customized exception.
- **Abstract Class, Sealed Class, Interface**
  - **PYQ:** What are the needs for partial class and sealed class? How do you relate delegate with events?
- **Delegate and Events**
  - **PYQ:** Delegates and Events
- **Partial Class**
  - **PYQ:** What are the needs for partial class and sealed class? How do you relate delegate with events?
- **Collections and Generics**
  - **PYQ 1:** Distinguish between collection and generics.
  - **PYQ 2:** Write a program to demonstrate the concept of collection and generics.
  - **PYQ 3:** Differentiate between generic and non-generic collections. Write a simple program to create generic class with generic constructor, generic member variable, generic property and generic method.
- **File I/O**
  - **PYQ:** File I/O operations in C#
- **LINQ (Language Integrated Query) Fundamentals: Lambda Expressions**
  - **PYQ 1:** Lambda Expression
  - **PYQ 2:** LINQ
- **Try Statements and Exceptions**
  - **PYQ:** Distinguish between collection and generics. What are named and positional attribute parameters? Write a program to create your own exception when the user gives subject name other than "C#".
- **Attributes: Attribute Classes, Named and Positional Attribute Parameters, Attribute Targets, Specifying Multiple Attributes**
  - **PYQ:** Distinguish between collection and generics. What are named and positional attribute parameters? Write a program to create your own exception when the user gives subject name other than "C#".
- **Asynchronous Programming: Principle of Asynchrony, Async/Await patterns in C#**
  - **PYQ:** Explain the async/await pattern in C#. Why is asynchronous programming important in web applications? Provide a simple example.

Unit 2: Introduction to ASP.NET (3 Hrs)

- **.NET, .NET Core, and Mono Frameworks**
  - *PYQ:* Differentiate between .NET, .NET Core, and Mono frameworks. Explain the compilation and execution process of .NET applications covering CLI, MSIL, and CLR.
- **ASP.NET Frameworks**
  - ASP.NET Web Forms (short notes asked)
  - ASP.NET MVC (no qn asked)
  - ASP.NET Web API (no qn asked)
  - ASP.NET Core (no qn asked)
- **.NET Architecture and Design Principles**
  - *PYQ:* Describe the .NET architecture design and principles.
- **Compilation and Execution of .NET Applications**
  - CLI, MSIL, and CLR (covered in PYQ 1)
- **.NET Core in Detail**
  - Build, run, test, and deploy applications using .NET CLI
    - *PYQ:* Explain the procedure for building, running, and deploying .NET Core applications.
    - *PYQ:* Describe the procedure of deploying .NET Core application.
    - *PYQ:* State the steps to deploy .NET Core application.
    - *PYQ:* How do you test .NET Core applications?

Introduction to .NET Framework

- **Definition:**
  .NET is a **software framework** created by **Microsoft** to make it easier to develop applications.
- **History:**
  The first version, **.NET Framework 1.0**, was released in **2002**.
- **Purpose:**
  .NET is used to develop different types of applications:
  - **Windows Forms Applications** – Desktop apps with graphical interfaces.
    Example: A simple calculator program for Windows.
  - **Web-based Applications** – Apps that run in a browser.
    Example: Online shopping website like Amazon.
  - **Web Services** – Services that allow apps to communicate over the internet.
    Example: A weather API that provides real-time weather info to different apps.
- **Programming Languages Supported:**
  .NET supports **more than 60 programming languages**, including:
  - **C#.NET** – Popular for Windows apps and web development.
  - **VB.NET** – Often used for business applications.
  - **C++.NET** – Used for performance-intensive apps.
  - **JScript.NET** – Used for scripting tasks.

Components of .NET Framework (Architecture)

The **.NET Framework** is made up of several components that work together to develop and run applications:

1. Applications Layer

This layer is where different types of applications are built:

1. WinForms (Windows Forms Applications)

- **What it is:** Used to make desktop programs with buttons, textboxes, and other controls.
- **Features:**

    - Drag-and-drop controls (easy GUI design)
    - Event-driven programming (responds to clicks, typing)
    - Can connect to databases easily
- **Simple Example: Student Attendance System**
    - Admin adds students, marks attendance, and views reports
    - Made using WinForms (C#) + SQL Server for data

2. ASP.NET (Web Applications)

- **What it is:** Used to create websites and web apps.
- **Features:**
    - Uses MVC (Model-View-Controller) design
    - Connects to databases using ADO.NET or EF Core
    - Handles login, sessions, and permissions
- **Simple Example: Online Library System**
    - Users can search and borrow books online
    - Admin can manage books and user accounts
    - Built using ASP.NET Core MVC + Entity Framework

3. ADO.NET (Database Connectivity)

- **What it is:** Used to connect programs to databases. Lets you read, add, update, and delete data.
- **Key Parts:**
    - **Connection:** Link to the database
    - **Command:** Run SQL queries
    - **DataReader:** Read data quickly, one row at a time
    - **DataAdapter & DataSet:** Work with data without staying connected to the database
- **Simple Example: Movie Database App**

2. Framework Class Library (FCL)

- A collection of **reusable classes, methods, and objects**.
- Works with **CLR** to provide functionality like file handling, string operations, networking, and error handling.
- Similar to **C++ header files** or **Java packages**.
- Installing .NET Framework installs both **FCL and CLR**.

Example: Using System.IO classes to read or write files.

3. Common Language Runtime (CLR)

- Acts like a **virtual machine** for .NET applications.
- Executes code from all .NET languages (C#, VB.NET, etc.) according to **common rules (CTS/CLS)**.
- Handles **memory management, object creation, data types, and exception handling**.

Example: Declaring a variable, creating an object, or handling a runtime error.

4. Integrated Development Environment (IDE)

- The **tool** for writing, debugging, and testing applications.
- *Example:* Visual Studio.

Compilation and Execution in .NET

1. **Compilation**
    - Code written in any .NET language is first compiled into **Intermediate Language (IL)**.
    - IL is **understandable by CLR** and stored as an **assembly**.
    - Two types of compilation:

    - **Debug:** Assembly is generated without optimization (used for testing).
    - **Release:** Assembly is fully optimized without debug symbols (used for production).
2. **Execution**
    - CLR's **Just-In-Time (JIT) compiler** converts IL into **binary code** that the machine can execute.
    - The program is then loaded into memory and runs.

Example: Opening Notepad triggers CLR to execute the binary code generated from IL.

Example Scenario

Suppose we want to create a **desktop calculator** using **C# Win Forms**.

Flow of a .NET Application

1. **Write Code (IDE)**
    - You open **Visual Studio** (IDE) and write C# code for the calculator.
    - Example: Buttons for numbers, addition, subtraction.
2. **Compilation**
    - Your C# code is compiled into **Intermediate Language (IL)**.
    - IL is stored as an **assembly (.exe or .dll)**.
    - This process checks for **syntax errors**.
3. **Execution (CLR + JIT)**
    - When you run the program, **CLR** loads the assembly.
    - **JIT compiler** converts IL into **machine code** (binary) your computer understands.
    - CLR handles **memory, objects, and exceptions** while the program runs.
4. **Using Framework Class Library (FCL)**
    - The calculator program uses FCL classes like System.Math to perform calculations.
    - Example: Math.Pow() to calculate powers.
5. **Output**
    - The calculator window opens.
    - You click buttons, perform operations, and see results.

.NET Framework Design Principles

1. Interoperability

- Allows .NET applications to work with existing code written in other languages (e.g., COM, native APIs).
- Ensures seamless integration of legacy components with new .NET applications.
- Example: A .NET program can use an older COM library for printing without rewriting it.

2. Memory Management

- Managed by the **Common Language Runtime (CLR)**.
- Automatic **garbage collection** frees unused memory, reducing memory leaks.
- Example: When an object is no longer used, CLR automatically releases its memory.

3. Portability

- .NET Core and later versions support **cross-platform development**.
- Applications can run on Windows, Linux, and macOS without major code changes.
- Example: A console application developed on Windows can run on Linux with .NET Core.

4. Security

- Built-in security features include **code access security** and **role-based security**.
- CLR ensures type safety and prevents unsafe operations.
- Example: Preventing unauthorized access to files or sensitive methods.

5. Simplified Development

- Provides a large **Base Class Library (BCL)** with ready-made functions for file I/O, database access, XML, etc.
- Supports **object-oriented programming**, reducing complexity.
- Example: Instead of writing custom code to connect to a database, use ADO.NET classes.

ASP.NET (Web Applications)

- **Definition:**
  ASP.NET is a free web framework created by **Microsoft** to build websites and web applications using **HTML, CSS, and JavaScript**.
- **Relation to .NET:**
  - ASP.NET is a **subset of the .NET Framework**.
  - It is the **successor of Classic ASP**.
  - While .NET is for **Windows, Web, and Server applications**, ASP.NET focuses on **web-based applications**.
- **Purpose:**
  - Makes creating **dynamic web pages** easier.
  - Can handle **server-side programming** (processing on the server) and **client-side programming** (interacting with the browser).
  - Mainly used for **business and enterprise web applications** on the Windows platform.
- **Programming Languages:**
  - Can use **C#** or **VB.NET**.
  - Any .NET language that compiles to **CIL (Common Intermediate Language)** can be used.
- **Key Points:**
  - ASP.NET runs on the **server**, and sends the final output (HTML, CSS, JS) to the **browser**.
  - Simplifies tasks like **authentication, session management, and database connections**.
- **Example:**
  - **Online Library System:**
    - Users can search, borrow, and return books online.
    - Admin can add or remove books and manage accounts.
    - Built using **ASP.NET with C# and SQL Server**.

Feature	.NET Framework	ASP.NET
Purpose	Develop Windows, Web, and Server applications	Develop web applications and dynamic websites
Scope	Broad framework for all types of applications	Subset of .NET focused on web development
Programming	Supports multiple languages (C#, VB.NET, C++, etc.)	Typically C# or VB.NET for web applications
Execution	Runs on CLR for desktop, server, and web apps	Runs on server to generate web pages for browsers
Use Case Example	Notepad (WinForms), Windows Services	Online Library Management System

.NET Core Overview:

- **What it is:** A new version of the .NET framework, designed to be modular, fast, lightweight, and cross-platform.
- **Cross-platform:** Works on **Windows, macOS, and Linux**.
- **Purpose:** Can be used to build **web, desktop, mobile, cloud, IoT, game, and machine learning applications**.
- **Languages supported: C#, F#, Visual Basic**.

- **Open-source:** Maintained by Microsoft and available on GitHub.

Key Characteristics:

1. **Open-source framework:** Free and maintained publicly; anyone can contribute.
2. **Cross-platform:** Same code can run on different operating systems.
3. **Consistent across architectures:** Works the same on **x64, x86, and ARM** processors.
4. **Wide-range of applications:** Suitable for almost all types of software development.
5. **Compatibility:** Works with older .NET Framework and Mono APIs using **.NET Standard**.

Example:

- You can build a **cloud-based chat application** using **ASP.NET Core** that runs on **Windows servers, Linux servers, or even on Mac development machines**, and users can access it via any browser.

ASP.NET Web Forms

- **What it is:** Part of the ASP.NET framework for building web pages that users access via a browser.
- **How it works:** Pages are written using **HTML, server controls, scripts, and server-side code**. When requested, the server runs the code, creates HTML, and sends it to the browser.
- **Features:**
  - **Server Controls:** Like buttons, text boxes, calendars; some can connect directly to databases.
  - **Master Pages:** Create a common layout for all pages in your site.
  - **Working with Data:** Can store, retrieve, and display data in tables, dropdowns, etc.
  - **State Management:** Keeps track of user data across pages and sessions.
  - **Security:** Helps protect your application from attacks.
  - **Performance:** Optimized for page processing, data handling, and server controls.
- **Example:** An **online student registration page** where students can register, and data is saved to a database.

ASP.NET MVC

- **What it is:** Model-View-Controller (MVC) framework for building dynamic websites. Works with ASP.NET Core too.
- **How it works:** Follows the **MVC pattern**:
  - **Model:** Handles data and business logic.
  - **View:** Displays data to users (UI).
  - **Controller:** Handles user requests and updates Model/View.
- **Advantages:** Fast development, clean separation of code, full control over HTML.
- **Example:** An **online shopping site** where product info (Model) is shown on pages (View), and user actions like adding to cart are handled by the Controller.

ASP.NET Web API

- **What it is:** A tool to make web services that send data to apps or websites.
- **How it works:** Sends data (like JSON) instead of web pages, so apps can use it easily.
- **Example:** A weather app gets temperature and forecast from a Web API.

Compilation and Execution of .NET Applications

- **Overview**:
  .NET applications go through multiple steps before they run. The process ensures code is language-independent, secure, and optimized for execution on any system with .NET runtime.
- **Steps of Compilation and Execution**:
  1. **Source Code**
     - Written in any .NET-supported language (C#, VB.NET, F#).

    - Example: C# code in .cs file.
2. **Compilation to Intermediate Language (IL/MSIL)**
    - The source code is compiled by a language-specific compiler into **Microsoft Intermediate Language (MSIL)**, sometimes called IL.
    - MSIL is platform-independent, which allows .NET applications to run on any machine with CLR.
3. **Assembly Creation**
    - The compiled IL is stored in an **assembly** (EXE or DLL).
    - Each assembly contains:
        - **Manifest**: Metadata about the assembly (version, culture, references).
        - **IL Code**: Compiled intermediate code.
4. **Just-In-Time (JIT) Compilation**
    - When the program runs, CLR compiles the IL into **native machine code** suitable for the operating system and CPU.
    - Types of JIT:
        - **Pre-JIT**: Compiles the complete IL at once before execution.
        - **Econo-JIT**: Compiles methods only when called, saving memory.
        - **Normal JIT**: Compiles methods as they are invoked for the first time.
5. **Execution by CLR**
    - CLR manages execution, memory allocation, garbage collection, and exception handling.
    - Ensures type safety and security.

- **Key Points**:
    - .NET allows multi-language development because all languages compile into the same IL.
    - Security and memory management are handled automatically by CLR.
    - Performance is optimized by JIT compilation.

.NET Architecture

1. Languages Layer (Top Layer)

- Contains .NET programming languages like **C#**, **VB.NET**, **JScript.NET**, etc.
- Developers write code using these languages.

2. Framework Layer

- **Common Language Specification (CLS):** Rules that all .NET languages must follow.
- **Common Type System (CTS):** Defines how data types are stored and handled in memory.
- **.NET Framework Class Library (FCL):** Ready-made classes and functions for building apps (desktop, web, mobile).
- **Application Models/Features:** Tools to build apps, like **ASP.NET**, **Windows Forms**, **Web Services**, **ADO.NET**, **Console apps**, etc.

3. Runtime Layer

- **Common Language Runtime (CLR):** The "engine" of .NET that executes code.
    - Handles **memory management (garbage collection)**, **security**, and **error handling**.
    - Converts intermediate code into machine code using **Just-in-Time (JIT) compiler**.

4. Base Layer

- **Common Language Infrastructure (CLI):** Platform that allows .NET apps to run independently of the programming language.
- Works on top of the **Operating System**.

Key Components (Simple Terms)

Component	Role
CLR	Runs all .NET programs, manages memory, security, and errors.
CLI	Language-independent platform for apps; supports performance, security, and exception handling.

CTS Standardizes data types for all .NET languages.

CLS Rules for language compatibility under .NET.

FCL Provides classes, functions, and tools to build different types of apps.

Compilation and Execution of .NET Applications

1. Writing the Code

You first write code in any .NET language like **C#**, **VB.NET**, or **F#**.
All these languages follow the **Common Type System (CTS)** rules, so they can work together.
Example:
You write a C# class that calls a method written in VB.NET.
This works because both follow the same CTS rules.

2. Compiling the Code → MSIL/IL

When you compile your .NET program, it does **not** become machine code directly.
Instead, the compiler converts it into:

MSIL (Microsoft Intermediate Language) / IL

- It is **platform-independent**.
- All .NET languages convert their code into IL.
- This is why **language interoperability** is possible.

Example:
C# code → IL
VB.NET code → IL
Both IL codes can call each other inside the same assembly.

3. Assemblies

The IL code is stored inside **assemblies** (.exe or .dll).
One assembly can contain **multiple modules**, written in **different .NET languages**.
All modules can reference each other as if written in the same language.

4. Execution by the CLR

When you run the program, the **Common Language Runtime (CLR)** takes over.
CLR does these tasks:

- Converts **IL into machine code** using Just-In-Time (JIT) compiler
- Manages memory
- Handles exceptions
- Provides security

Code executed by the CLR is called **managed code**.
Code that runs directly on the machine without CLR is called **unmanaged code**.

Mono

1. Early 2000s – The Problem

When Microsoft first released the **.NET Framework**, it was powerful but had one big limitation:
It ran only on Windows.
Developers who wanted to use C# or .NET on **Linux, macOS, or mobile devices** had no option.

2. 2001–2004 – The Birth of Mono

A group of developers at a company called **Xamarin** decided to solve this.
They started an open-source project named **Mono**.
Their goal was simple:
Make .NET work everywhere, not just on Windows.
So they built:

- their own **CLR** (Mono runtime)

- their own **C# compiler**
- class libraries similar to Microsoft's .NET

Mono slowly became popular among Linux users and mobile developers.

3. 2010–2015 – Growing With Mobile and Games

Mono got two big uses:

- **Xamarin mobile apps** (Android/iOS apps using C#)
- **Unity game engine**, which used Mono internally

This made Mono famous worldwide.

4. 2016 – Microsoft Creates .NET Core

Microsoft realized developers needed a **modern, fast, cross-platform** .NET.
So they created **.NET Core**, which was:

- fully open-source
- much faster than Mono
- supported Windows, Linux, macOS
- officially maintained by Microsoft

At this point, Microsoft's .NET Core started replacing the need for Mono in most areas.

5. Today – How Things Stand

- **Mono is still used in Unity and some mobile frameworks**
- **.NET Framework** is now old and Windows-only
- **.NET Core (now just called .NET)** is the main future platform

How Mono differs from .NET Framework and .NET Core

Feature	Mono	.NET Framework	.NET Core / .NET (Modern)
Platform Support	Cross-platform (Linux, macOS, mobile)	Windows-only	True cross-platform (Windows, Linux, macOS)
Ownership	Originally Xamarin (open source)	Microsoft	Microsoft (open-source)
Use Case	Unity, mobile, legacy cross-platform apps	Desktop, enterprise Windows apps	Modern web, cloud, APIs, cross-platform apps
Performance	Good for mobile but slower than .NET Core	Limited to Windows	Very high performance, optimized
Future	Mostly used for Unity and legacy	No longer developed (maintenance only)	Current and actively developed

Basic Language Constructs (C#)(never asked)

These are the basic building blocks used to write any C# program.

1. Variables

Used to store data in memory.
Example:

```
int age = 20;
```

2. Data Types

Two types:

- **Value types**: int, float, bool, char
- **Reference types**: string, arrays, class, object

3. Operators

Used to perform operations.

Example: +, -, *, /, ==, !=

4. Control Statements

Used to control program flow:

- **if/else**
- **switch**
- **for, while, do-while loops**

Example:

```
for (int i = 1; i <= 5; i++)
{
   Console.WriteLine(i);
}
```

Constructor

A constructor is a **special method** inside a class that runs automatically whenever an object is created.

Features:

- Same name as the class
- No return type
- Used to initialize (set starting values of) objects

Example:

```
class Student
{
   public string name;
public Student()  // constructor
   {
      name = "Unknown";
   }
}
```

Creating an object:

```
Student s = new Student(); // constructor runs here
```

Properties(asked with indexer)

Properties are **special class members** used to access private fields safely.
They work like variables but internally use methods (get/set).

Why needed?

To control how data inside a class is read or changed.

Example:

```
class Person
{
   private int age;
public int Age
   {
      get { return age; }
      set { age = value; }
   }
}
```

Using it:

```
Person p = new Person();
```

```
p.Age = 25;         // set
Console.WriteLine(p.Age);  // get
```

Arrays

An array is a **collection of same-type items stored in continuous memory**.

Example:

1. Declare an array first, then assign values later

```
int[] nums = new int[3];
nums[0] = 10;
nums[1] = 20;
nums[2] = 30;
```

2. Declare and initialize at the same time

```
int[] nums = { 10, 20, 30 };
```

3. Using new keyword with values

```
int[] nums = new int[] { 10, 20, 30 };
```

4. Declare array without size, then assign

```
int[] nums;
nums = new int[] { 10, 20, 30 };
```

5. Multi-dimensional array (2D array)

```
int[,] grid = {
   { 1, 2, 3 },
   { 4, 5, 6 }
};
```

Key points:

- Index starts from 0
- Fixed size once created

String

A string is a **sequence of characters** (text).
It is a **reference type** but behaves like a value type because it is **immutable** (cannot be changed after creation).

Example:

```
string name = "Aashish";
string full = name + " Panta";
```

Indexers in C# (asked in exam)

Indexers let an object be **accessed like an array** using the **[ ]** brackets.

They allow you to use:

obj[index]
instead of calling methods or properties manually.
Indexers are mainly used when your class represents a **collection**.

Difference Between Indexers and Properties

Indexer	**Property**
Lets you access object like an **array** using []	Accessed by name, no []
Usually takes one or more **parameters** (index)	No parameters
Used when class represents **multiple values**	Used to access a **single value**
Syntax: this[index]	Syntax: property name

Syntax :

```
public DataType this[int index]   // indexer declaration
  {
     get
     {
        return array[index];      // return value at given index
     }
     set
     {
        array[index] = value;    // set value at given index
     }
  }
```

Example: Properties vs Indexers

```
class StudentMarks
{
   private int[] marks = new int[3];   // store 3 subject marks
// Property example: access first subject mark
   public int Math
   {
      get { return marks[0]; }
      set { marks[0] = value; }
   }
// Indexer example: access all marks using index
   public int this[int index]
   {
      get { return marks[index]; }
      set { marks[index] = value; }
   }
}
```

Using Properties

```
StudentMarks student = new StudentMarks();
student.Math = 85;              // set math mark using property
Console.WriteLine(student.Math);  // get math mark → 85
```

Note: Property is good when you want to **access a single value by name**.

Using Indexers

```
StudentMarks student = new StudentMarks();
student[0] = 85;   // set math mark using indexer
student[1] = 90;   // set science mark
student[2] = 78;   // set english mark
Console.WriteLine(student[0]);   // get math mark → 85
Console.WriteLine(student[1]);   // get science mark → 90
Console.WriteLine(student[2]);   // get english mark → 78
```

Note: Indexer is useful when you want the class to **behave like an array** and access multiple values by index.

Inheritance

- **Definition:** Inheritance allows a class (child) to reuse properties and methods of another class (parent).
- **Purpose:** Helps avoid code duplication and supports reusability.

Syntax Example:

```
class Parent
{
   public void Greet()
   {
      Console.WriteLine("Hello from Parent");
   }
}
class Child : Parent   // Child inherits Parent
{
   public void SayHi()
   {
      Console.WriteLine("Hi from Child");
   }
}
```

Usage:

```
Child c = new Child();
c.Greet();  // inherited from Parent
c.SayHi();  // from Child
```

base Keyword

- **Definition:** Refers to the parent class from inside the child class.
- **Use:** Call parent class methods or constructors from the child class.

Example:

```
class Child : Parent
{
   public void Show()
   {
      base.Greet();  // calls Parent's Greet method
   }
}
```

Key Point:
Use base when you want to access or override something from the parent class.

Method Hiding vs Method Overriding

Both are ways a **child class** can change or replace behavior of a **parent class method**, but they work differently.

1. Method Hiding (new keyword)

- The child class **hides** the parent's method.
- The parent method still exists, but the child's version is used **only when called through the child class object**.
- It does **not** replace the parent's method in the parent class reference.

Example:

```
class Vehicle
{
   public void Start()
   {
      Console.WriteLine("Vehicle started");
   }
}
class Car : Vehicle
{
   public new void Start() // hides the parent method
   {
      Console.WriteLine("Car started");
   }
}
Vehicle v = new Vehicle();
v.Start();  // Output: Vehicle started
Car c = new Car();
c.Start();  // Output: Car started
Vehicle v2 = new Car();
v2.Start(); // Output: Vehicle started (parent version is called!)
```

2. Method Overriding (override keyword)

- The child class **replaces** the parent's method.
- The parent method must be marked as virtual.
- The child's method is used **even if the object is referenced as a parent**.

Example:

```
class Vehicle
{
   public virtual void Start()
   {
      Console.WriteLine("Vehicle started");
   }
}
class Car : Vehicle
{
   public override void Start() // overrides the parent method
   {
      Console.WriteLine("Car started");
   }
}
Vehicle v = new Vehicle();
v.Start();  // Output: Vehicle started
```

```
Car c = new Car();
c.Start();  // Output: Car started
Vehicle v2 = new Car();
v2.Start(); // Output: Car started (child version is called!)
```

Now, Car doesn't just hide Vehicle's method—it **replaces** it. No matter if you look through Vehicle or Car, the Car behavior is used.

Quick Table:(asked in exam)

Feature	**Method Hiding (new)**	**Method Overriding (override)**
Parent method	non-virtual	virtual
Child method	new	override
Parent reference	Calls parent version	Calls child version
Behavior	Only changes when using child	Always uses child version

(Short note asked)

Polymorphism

Polymorphism means **one thing can take many forms**. In C#, it allows a **base class reference** to work with different derived class objects. This helps make code flexible and easy to extend.

Example:

```
class Vehicle
{
   public virtual void Start()
   {
      Console.WriteLine("Vehicle started");
   }
}
class Car : Vehicle
{
   public override void Start()
   {
      Console.WriteLine("Car started");
   }
}
class Bike : Vehicle
{
   public override void Start()
   {
      Console.WriteLine("Bike started");
   }
}
// Using polymorphism
Vehicle v1 = new Car();
Vehicle v2 = new Bike();
v1.Start();  // Output: Car started
v2.Start();  // Output: Bike started
```

Explanation:

- Vehicle is the base class.
- Car and Bike are derived classes.
- We can use Vehicle reference for any vehicle type.

- The correct method is called depending on the actual object (Car or Bike).

Structs and Enums

1. Structs:

- A struct is a **value type** that can store data and methods.
- Useful for small objects that don't require inheritance.
- Stored on **stack memory**.

Example:

```
struct Point
{
   public int X;
   public int Y;
}
Point p1;
p1.X = 10;
p1.Y = 20;
Console.WriteLine($"Point: ({p1.X}, {p1.Y})");
```

2. Enums:

- An enum is a **special type** to define **named constants**.
- Makes code readable and prevents using magic numbers.

Example:

```
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday };
Days today = Days.Wednesday;
Console.WriteLine(today);  // Output: Wednesday
```

Difference between Struct and Enum(pyq):

Feature	Struct	Enum
Definition	A value type that can hold data members (fields, properties) and methods.	A special value type used to define a set of named constants.
Purpose	To represent a small data structure like a point, date, or complex number.	To give meaningful names to a set of related constant values.
Members	Can have fields, methods, properties, constructors, etc.	Only has named constants (no methods or properties).
Example	struct Point { public int X; public int Y; }	enum Days { Sunday, Monday, Tuesday }
Usage	When you need a lightweight object to store data.	When you need a list of fixed, related values for readability and maintainability.

Abstract Class, Sealed Class, and Interface in C#:

Asked in exams

1. Abstract Class:

- A class that **cannot be instantiated** directly.
- Used when you want to provide **common base functionality** but still force derived classes to implement certain methods.
- Can have methods with or without implementation.

Example:

```
abstract class Vehicle
{
   public abstract void Start(); // Must be implemented in derived class
   public void Stop() { Console.WriteLine("Vehicle stopped"); } // Optional implementation
}
class Car : Vehicle
{
   public override void Start() { Console.WriteLine("Car started"); }
}
```

Use: When you want a base class that **defines rules** but cannot be used alone.

2. Sealed Class:

- A class that **cannot be inherited**.
- Used to **prevent further derivation**, for security or to stop changes to its behavior.

Example:

```
sealed class Calculator
{
   public int Add(int a, int b) { return a + b; }
}
```

Use: When you want a **final class** that shouldn't be extended.

3. Interface:

- A **contract** that defines **methods/properties** but provides **no implementation**.
- Any class implementing the interface must provide implementations for all its members.

Example:

```
interface IDrive
{
   void Accelerate();
}
class Bike : IDrive
{
   public void Accelerate() { Console.WriteLine("Bike is accelerating"); }
}
```

Use: When you want **different classes to share the same behavior** but implement it differently.

Feature	**Abstract Class**	**Interface**
Methods	Can have both implemented & abstract	Usually only method declarations
Fields/Properties	Can have fields and properties	Cannot have fields (properties okay)
Constructor	Yes	No
Multiple Inheritance	No	Yes
Use Case	Common behavior + enforced rules	Only enforce rules/contract

(asked in exam)

Delegates

- Think of a delegate as a **reference or pointer to a method**.
- It allows you to **store a method inside a variable** and call it later.
- Delegates are useful when you want your program to **decide at runtime which method to call**, rather than hardcoding it.
- **Analogy:** Imagine a delegate as a **remote control**. You can assign any TV (method) to it and press the button to run that TV (method).

Events

- An event is a **special type of delegate** used to **signal that something has happened**.
- Only the class that defines the event can **trigger it**, but other classes can **subscribe to it** and respond.
- **Analogy:** Think of an event as a **doorbell**. The homeowner (class that owns the event) rings it, but anyone in the house (subscribers) can hear it and respond.

```
// Delegate declaration
public delegate void Notify(string message);

// Event declaration using the delegate
public event Notify OnNotify;

// Using the delegate/event
OnNotify("Task completed!");
```

Explanation:

- Notify is a delegate type that can point to any method taking a string and returning nothing (void).
- OnNotify is an event of type Notify. Only the class that declares it can trigger it.
- When OnNotify("Task completed!") is called, all subscribed methods (listeners) are executed.

Key Point:

Delegate = reference to a method.
Event = a controlled way for others to react to an action using a delegate.

How They Are Related

- Delegates are the **foundation of events**.
- Events **use delegates internally**, but they add a layer of **control**—ensuring that only the owner can trigger the action, while others can only listen and react.

Imagine a **school bell system**:

- The **delegate** is like the **bell signal**—it knows what kind of actions it can notify (like ringing a bell).
- The **event** is the **actual ringing of the bell**. Only the school administration can trigger it.
- **Subscribers** are the **students and teachers** who respond when the bell rings.

Partial Class:

Asked in exam
A **partial class** allows a single class to be split into multiple files. All parts are combined by the compiler to form one complete class when the program runs.

Why we need it:

1. **Organizing code:** Splitting a big class into multiple files makes it easier to read and maintain.
2. **Team work:** Different developers can work on different parts of the same class without conflicts.
3. **Auto-generated code:** Tools like Visual Studio can generate part of a class automatically. Partial classes let you keep your custom code separate.

Example:

```
// File1.cs
```

```
public partial class Student
{
   public string Name;
   public int Age;
}
// File2.cs
public partial class Student
{
   public void DisplayInfo()
   {
      Console.WriteLine($"Name: {Name}, Age: {Age}");
   }
}
// Using the class
Student s = new Student();
s.Name = "Aashish";
s.Age = 20;
s.DisplayInfo();
```

Explanation:
Even though Student is split into two files, the compiler combines them into one class. You can access both fields and methods normally.

COLLECTIONS AND GENERICS

1. What are Collections?

- A **collection** is like a bag where you can store multiple items.
- Non-generic collections can store **any type of item** (int, string, object, etc.), but this can be risky because you may need to convert types before using them.
- Example:

```
using System;
using System.Collections;
class Program
{
   static void Main()
   {
      ArrayList bag = new ArrayList(); // Non-generic collection
      bag.Add(10);     // int
      bag.Add("Hello"); // string
foreach (var item in bag)
      {
         Console.WriteLine(item);
      }
   }
}
```

Story:

- ArrayList is a bag.
- You can put anything in it.
- Problem: If you want to do math on the first item (10), you have to convert it from object to int.

2. What are Generics?

- A **generic** is a **type-safe collection**.
- You tell it what type it will store, and it **cannot accept other types**.

- Safer, faster, and avoids type conversion errors.

Example:

```
using System;
using System.Collections.Generic;
class Program
{
   static void Main()
   {
      List<int> numbers = new List<int>(); // Generic collection
      numbers.Add(10);
      numbers.Add(20);
foreach (int num in numbers)
      {
         Console.WriteLine(num);
      }
// numbers.Add("Hello"); // ❌ Error! Only integers allowed
   }
}
```

Story:

- List<int> is a bag for integers only.
- No need to convert types.

3. Difference Between Collections and Generics (PYQ 1)

Feature	Non-generic Collection	Generic Collection
Type Safety	Can store any type (not safe)	Stores only specified type
Performance	Slower (boxing/unboxing)	Faster (no type conversion)
Errors	Runtime errors possible	Compile-time errors prevented
Example	ArrayList	List<int>

Write a simple program to create generic class with generic constructor, generic member variable, generic property and generic method.

```
using System;
// Step 1: Create a generic class
class Bag<T>
{
   // Generic member variable
   private T item;
// Generic constructor
   public Bag(T value)
   {
      item = value;
   }
// Generic property
   public T Item
   {
      get { return item; }
      set { item = value; }
   }
// Generic method
   public void ShowItem()
   {
```

```
        Console.WriteLine("Item: " + item);
    }
}
class Program
{
    static void Main()
    {
        // Using generic class with string type
        Bag<string> stringBag = new Bag<string>("Hello World");
        stringBag.ShowItem();          // Output: Item: Hello World
        stringBag.Item = "C# is fun";   // Using property to set value
        Console.WriteLine(stringBag.Item); // Output: C# is fun
// Using generic class with int type
        Bag<int> intBag = new Bag<int>(100);
        intBag.ShowItem();             // Output: Item: 100
        intBag.Item = 200;             // Using property to set value
        Console.WriteLine(intBag.Item);  // Output: 200
    }
}
```

Explanation

1. Generic Class

class Bag<T>

- T is a **placeholder** for any type.
- When we create an object, we tell it what T should be (like int or string).

2. Generic Member Variable

private T item;

- This variable can hold any type of value, depending on what we pass when creating the object.

3. Generic Constructor

```
public Bag(T value)
{
    item = value;
}
```

- The constructor **accepts a value of type T** and stores it in the member variable.
- Allows initializing the object with any type.

4. Generic Property

```
public T Item
{
    get { return item; }
    set { item = value; }
}
```

- Property lets us **get** or **set** the value of item.
- Works for any type because it's generic.

5. Generic Method

```
public void ShowItem()
{
    Console.WriteLine("Item: " + item);
}
```

- Method shows the value of item.
- Can handle any type because item is generic.

6. Using the Generic Class

```
Bag<string> stringBag = new Bag<string>("Hello World");
Bag<int> intBag = new Bag<int>(100);
```

- stringBag can store strings only.
- intBag can store integers only.
- No type conversion is needed, and it's **safe**.

FILE I/O in C#

What is File I/O?

- File I/O (Input/Output) allows your program to **read from** and **write to** files on the computer.
- Example: saving student marks to a text file or reading data from a file.

Common Classes for File I/O:

1. File – simple methods like File.ReadAllText(), File.WriteAllText().
2. StreamReader – reads text from files line by line.
3. StreamWriter – writes text to files line by line.

Example:

```
using System;
using System.IO;
class Program
{
   static void Main()
   {
      string path = "data.txt";
// Write text to file
      File.WriteAllText(path, "Hello File I/O!");
// Read text from file
      string content = File.ReadAllText(path);
      Console.WriteLine(content); // Output: Hello File I/O!
   }
}
```

Explanation:

- File.WriteAllText() writes content to the file.
- File.ReadAllText() reads content from the file.
- Easy way to store and retrieve data from files.

LINQ (Language Integrated Query) Fundamentals

What is LINQ?

- LINQ is a **way to query collections** like arrays, lists, or databases in C# using **SQL-like syntax**.
- Makes working with data very easy and readable.

1. Lambda Expressions

- Lambda is a **shortcut way to write a method inline**.
- Syntax: (input) => expression

Example:

```
int[] numbers = { 1, 2, 3, 4, 5 };
// Lambda expression to get even numbers
var evenNumbers = Array.FindAll(numbers, n => n % 2 == 0);
foreach (var num in evenNumbers)
{
   Console.WriteLine(num); // Output: 2, 4
```

```
}
```
Explanation:

- n => n % 2 == 0 is the lambda expression.
- It checks each number n to see if it's even.
- Makes filtering or processing data easy.

2. LINQ Example

```
using System;
using System.Linq;
class Program
{
   static void Main()
   {
      int[] numbers = { 1, 2, 3, 4, 5 };
// LINQ query to select even numbers
      var evenNumbers = from n in numbers
                        where n % 2 == 0
                        select n;
foreach (var num in evenNumbers)
      {
         Console.WriteLine(num); // Output: 2, 4
      }
   }
}
```
Explanation:

- from n in numbers → loop through each number
- where n % 2 == 0 → filter only even numbers
- select n → choose the number to include in the result
- LINQ makes it **like writing a query on data** directly.

TRY STATEMENTS AND EXCEPTIONS

What are Exceptions?

- Exceptions are **errors that happen while a program is running**.
- Example: dividing by zero, accessing an invalid array index, or entering wrong input.
- If not handled, exceptions **crash the program**.

Why Handle Exceptions?

- To **prevent program from crashing**.
- To **inform the user** about the problem.
- To allow the program to **continue running safely**.

Try-Catch Block

- try → block of code where exceptions might occur.
- catch → block of code to handle the exception.
- Optional: finally → block of code that always runs, whether exception occurs or not.

Custom Exception for Divide by Zero

```
using System;
// Custom Exception
class DivideByZeroCustomException : Exception
{
   public DivideByZeroCustomException(string message) : base(message) { }
```

```
}
class Program
{
   static void Main()
   {
      try
      {
         Console.Write("Enter numerator: ");
         int numerator = int.Parse(Console.ReadLine());
        Console.Write("Enter denominator: ");
                int denominator = int.Parse(Console.ReadLine());
        if (denominator == 0)
                {
                   throw new DivideByZeroCustomException("Cannot divide by zero!");
                }
int result = numerator / denominator;
         Console.WriteLine("Result: " + result);
      }
      catch (DivideByZeroCustomException ex)
      {
         Console.WriteLine("Error: " + ex.Message);
      }
   }
}
```

Explanation:

1. DivideByZeroCustomException is a **custom exception class**.
2. When the user enters 0 as the denominator, we **throw this custom exception**.
3. The catch block handles it and **shows a friendly message**, instead of crashing the program.

Example Run:

```
Enter numerator: 10
Enter denominator: 0
Error: Cannot divide by zero!
```

ATTRIBUTES IN C#

What are Attributes?

- Attributes are **special metadata you can attach to classes, methods, properties, or other code elements**.
- They **provide additional information** to the compiler or runtime about how a program should behave.
- Think of it like **labels or tags** on your code.

Example:

```
[Obsolete("This method is outdated")]
void OldMethod() { }
```

- Here, [Obsolete] is an **attribute** that tells the compiler that the method is outdated.

Attribute Classes

- Every attribute is actually a **class that inherits from System.Attribute**.
- You can also **create your own custom attribute** by making a class that extends Attribute.

Example:

```
class MyAttribute : Attribute
{
   public string Info { get; set; }
}
```

1. Positional Parameters

- These are **required values passed directly to the attribute's constructor**.
- They **must be provided in the order defined** in the attribute class.
- Think of them as the "main information" the attribute needs.

Example:

```
[Obsolete("Use NewMethod instead")]  // "Use NewMethod instead" is positional
void OldMethod() { }
```

- Here, "Use NewMethod instead" is a **positional parameter**.

2. Named Parameters

- These are **optional values you set by name** when using the attribute.
- They **don't depend on order** and are more like "extra options."

Example:

```
[Obsolete("Use NewMethod instead", IsError = true)]
void OldMethod() { }
```

- IsError = true is a **named parameter**, giving extra instruction to the compiler.

Asynchronous Programming in C#

Principle of Asynchrony

- Asynchronous programming allows a program to **perform tasks without waiting for other tasks to complete**.
- Instead of blocking the program while waiting (like reading a file or fetching data from a web server), the program can continue doing other work.
- Improves **responsiveness** and **performance**, especially in **web apps** where users don't want to wait for long-running tasks.

Async/Await Pattern

- async marks a method as **asynchronous**.
- await tells the program to **wait for the task to complete**, but without blocking the rest of the program.
- Together, they make it easy to write **non-blocking, readable code**.

The code:

```
using System;
using System.Threading.Tasks;
class Program
{
   static async Task Main()
   {
      Console.WriteLine("Starting task...");
      await LongRunningTask();
```

```
    Console.WriteLine("Task finished!");
  }
static async Task LongRunningTask()
  {
    await Task.Delay(3000); // Simulates a 3-second task
    Console.WriteLine("Task completed after 3 seconds.");
  }
}
```

Step-by-step explanation

1. using System; and using System.Threading.Tasks;

- System namespace is needed for **basic things** like Console.WriteLine.
- System.Threading.Tasks is needed to use **Task**, which represents a unit of work that can run asynchronously.

2. class Program

- Defines a class named Program.
- In C#, all code must be inside a **class**.

3. static async Task Main()

- static means this method belongs to the class itself and can be called **without creating an object**.
- async tells C# that this method will use **await** inside it, i.e., it will do some asynchronous operations.
- Task is the **return type** for asynchronous methods that don't return any value (like void for normal methods).
    - If the method returned a value, we would write Task<int> or Task<string>.

4. Console.WriteLine("Starting task...");

- Prints "Starting task..." to the console.
- This happens **immediately** when the program runs.

5. await LongRunningTask();

- LongRunningTask() is called.
- await **pauses this line** until LongRunningTask finishes, **but it does not block the program thread**.
    - Without await, the program would call the method and move on immediately.
- This allows the program to **stay responsive** if this were a UI or web app.

6. Console.WriteLine("Task finished!");

- This line executes **after** LongRunningTask() completes.
- Shows "Task finished!" in the console.

7. static async Task LongRunningTask()

- Another asynchronous method.
- Marked with async because it will have an await inside.
- Returns a Task because it is asynchronous.

8. await Task.Delay(3000);

- Task.Delay(3000) creates a task that **waits for 3000 milliseconds (3 seconds)**.
- await pauses LongRunningTask until this delay completes.
- **Important:** The program thread is not blocked. The main method can continue running other tasks if there were any.

9. Console.WriteLine("Task completed after 3 seconds.");

- Prints a message **after 3 seconds**.

- This shows that the delay has finished and the asynchronous task is done.